Portfolio 1: Database Design and Implementation

Project: Stretto Technologies Database System

Database Environment: PostgreSQL (pgAdmin 4)

Data Volume: 50,000 Transactional Records

1. Chosen Case Study Description

Stretto Technologies is a Lagos-based e-commerce and electronics sourcing platform specializing in the procurement and sale of gadgets, including smartphones, laptops, and gaming consoles. The organization operates on a dynamic sourcing model, procuring inventory on-demand from established channels such as OA Plug Device, UK Direct Importers, and Gaming Hub NG, rather than holding traditional warehouse stock.

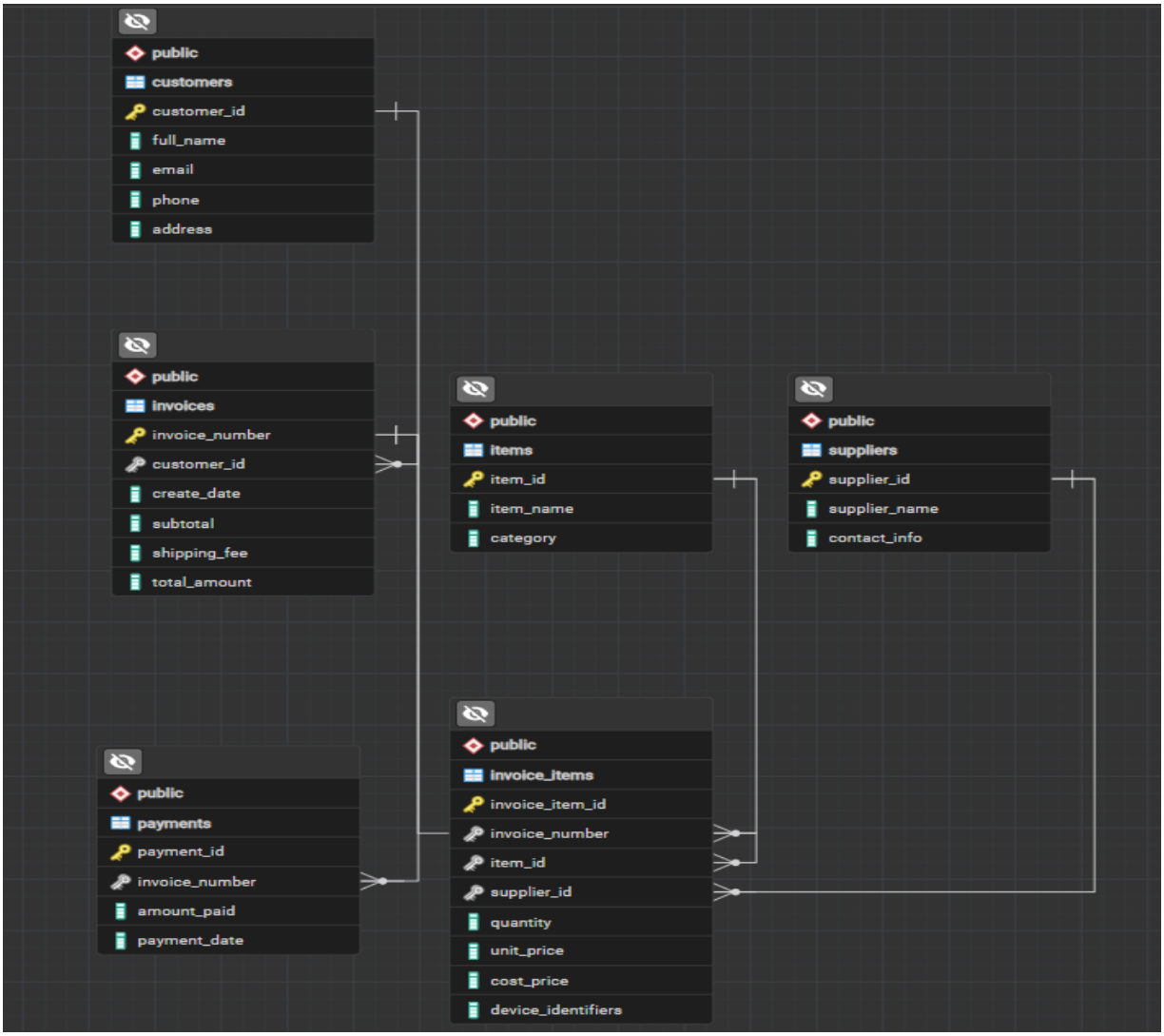
The primary users of this database system are the business administrators and procurement officers who require real-time visibility into sales, vendor performance, and customer payment statuses. The database stores critical operational data, including customer profiles, vendor directories, master product catalogs, structured invoices, granular line-item sourcing records, and partial or complete payment logs. The key operations this database supports include generating customer invoices, linking individual sold items to specific sourcing channels to calculate exact profit margins, tracking outstanding balances on partially paid invoices, and managing unique device identifiers (IMEIs and serial numbers) for warranty enforcement.

Stretto Technologies requires a robust relational database to manage its e-commerce platform and device sourcing operations. The system is designed to streamline core commercial transactions by tracking customer accounts, managing supplier records, cataloging stock items, and processing multi-item sales invoices, alongside providing a structured mechanism for recording and reconciling financial payments.

2. Entity-Relationship Diagram (ERD) Description

The Entity-Relationship Diagram (ERD) serves as the visual blueprint for the Stretto Technologies database, mapping out the logical structure and defining how data flows across the system. It provides a comprehensive overview of the relational architecture, illustrating the specific entities, their primary attributes, foreign key constraints, and the exact nature of their relationships. By defining these connections visually, the ERD ensures that data integrity rules are strictly enforced and prevents the creation of orphan records during transaction processing. Within this schema, several critical relationships are established using explicit primary and foreign key mapping to accurately model the device-sourcing business logic. The system utilizes one-to-many (1:N) cardinalities to reflect real-world commercial operations. For example, a single customer in the customers table can generate multiple purchase transactions over time,

creating a one-to-many relationship with the invoices table linked via the customer_id integer. Furthermore, because a single invoice can contain various purchased products, the invoices table shares a one-to-many relationship with the invoice_items junction table, linked securely by the invoice_number string. The supply chain and inventory components are also meticulously mapped through foreign key constraints within the junction table. The suppliers table connects to invoice_items (1:N) using supplier_id to track exactly which vendor sourced the specific items sold. The items table maintains a similar relationship with invoice_items via item_id to record every instance a specific product is purchased. Finally, to handle comprehensive financial reconciliation, the invoices table links to the payments table (1:N) via the invoice_number. This specific cardinality allows a single invoice to receive multiple partial payment installments over time until the outstanding balance is completely cleared.



3. Relational Schema, Keys, and Constraints

The database consists of 6 normalized tables. Keys and constraints are strictly enforced to maintain referential integrity across the system, utilizing specific data types to optimize storage and precision.

- Customers

✓ Primary Key: customer_id (SERIAL)
✓ Attributes: full_name (VARCHAR 255), email (VARCHAR 255), phone (VARCHAR 50), address (TEXT).
✓ Constraints: full_name is NOT NULL

- Suppliers

✓ Primary Key: supplier_id (SERIAL)
✓ Attributes: supplier_name (VARCHAR 255), contact_info (VARCHAR 255).
✓ Constraints: supplier_name is NOT NULL.

- Items

✓ Primary Key: item_id (SERIAL)
✓ Attributes: item_name (VARCHAR 255), category (VARCHAR 100).
✓ Constraints: item_name is NOT NULL.

- Invoices

✓ Primary Key: invoice_number (VARCHAR 50)
✓ Foreign Key: customer_id (INT) references customers.
✓ Attributes: create_date (TIMESTAMP), subtotal (DECIMAL 12,2), shipping_fee (DECIMAL 12,2), and total_amount (DECIMAL 12,2) for strict financial precision.
✓ Constraints: customer_id, create_date, subtotal, and total_amount are NOT NULL. shipping_fee enforces a DEFAULT 0.00 constraint.

- Invoice_items

✓ Primary Key: invoice_item_id (SERIAL)
✓ Foreign Keys: invoice_number references invoices, item_id references items, supplier_id references suppliers.
✓ Attributes: quantity (INT), unit_price (DECIMAL 12,2), cost_price (DECIMAL 12,2), device_identifiers (TEXT).
✓ Constraints: The three foreign keys, quantity, and unit_price are all NOT NULL. quantity enforces a DEFAULT 1 constraint.

- Payments

✓ Primary Key: payment_id (BIGINT)
✓ Foreign Key: invoice_number (VARCHAR 50) references invoices.
✓ Attributes: amount_paid (DECIMAL 12,2), payment_date (TIMESTAMP).
✓ Constraints: invoice_number, amount_paid, and payment_date are NOT NULL.

4. Normalisation Explanation

The original data architecture for Stretto Technologies consisted of flat, deeply nested JSON documents exported from an inventory application. While suitable for basic receipt generation, this structure suffered from severe data redundancy and update anomalies. The data was systematically decomposed into a 6-table relational schema (customers, suppliers, items, invoices, invoice_items, and payments) adhering to the strict rules of database normalization:

- **First Normal Form (1NF):** The raw JSON contained nested arrays for both purchased items and financial transactions within a single invoice record. To achieve 1NF and ensure atomicity, these arrays were extracted into dedicated invoice_items and payments tables. This ensures that every cell contains atomic (indivisible) values, and each row represents a single, unique line item or payment iteration.
- **Second Normal Form (2NF):** The schema satisfies 2NF by ensuring all non-key attributes are fully functionally dependent on the primary key. In the invoice_items bridge table, attributes such as quantity, unit_price, cost_price, and device_identifiers depend entirely on the explicit primary key (invoice_item_id), officially resolving the logical dependencies between the invoice and the specific item.
- **Third Normal Form (3NF):** The database was brought into 3NF by eliminating transitive dependencies. In the original JSON, customer details (name, email, phone, address) were repeated identically on every invoice. This data was isolated into a dedicated customers table, linked to the invoices table via a customer_id foreign key.

Similarly, sourcing channels were separated into a suppliers table. This structure prevents update anomalies; if a customer changes their address or a supplier updates their contact email, the modification only needs to occur in one central row, instantly propagating the correct data to all associated invoices.

5. Schema Implementation, Data Population, and Verification

The normalized schema was successfully implemented in PostgreSQL. The physical tables were created using the following data definition scripts, complete with explicit constraints to maintain referential integrity.

1. **Create Customers Table**

```
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(50),
    address TEXT
);
```

2**. Create Suppliers Table (Your Sourcing Channels)**

```
CREATE TABLE suppliers (
    supplier_id SERIAL PRIMARY KEY,
    supplier_name VARCHAR(255) NOT NULL,
    contact_info VARCHAR(255)
);
```

3. **Create Items Table (Master Catalog)**

```
CREATE TABLE items (
    item_id SERIAL PRIMARY KEY,
```

```
    item_name VARCHAR(255) NOT NULL,
    category VARCHAR(100)
);
```

4. **Create Invoices Table (The Core Transaction)**

```
CREATE TABLE invoices (
    invoice_number VARCHAR(50) PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customers(customer_id),
    create_date TIMESTAMP NOT NULL,
    subtotal DECIMAL(12, 2) NOT NULL,
    shipping_fee DECIMAL(12, 2) DEFAULT 0.00,
    total_amount DECIMAL(12, 2) NOT NULL
);
```

5. **Create Invoice_Items Table (The Bridge & Sourcing Record)**

```
CREATE TABLE invoice_items (
    invoice_item_id SERIAL PRIMARY KEY,
    invoice_number VARCHAR(50) NOT NULL REFERENCES invoices(invoice_number),
    item_id INT NOT NULL REFERENCES items(item_id),
    supplier_id INT NOT NULL REFERENCES suppliers(supplier_id),
    quantity INT NOT NULL DEFAULT 1,
    unit_price DECIMAL(12, 2) NOT NULL,
    cost_price DECIMAL(12, 2),
    device_identifiers TEXT
);
```

6. **Create Payments Table (Financial Tracking)**

```
CREATE TABLE payments (
    payment_id BIGINT PRIMARY KEY,
    invoice_number VARCHAR(50) NOT NULL REFERENCES invoices(invoice_number),
    amount_paid DECIMAL(12, 2) NOT NULL,
    payment_date TIMESTAMP NOT NULL
);
```

Data Population: To stress-test the database for operational readiness, high-volume mock data was injected using PostgreSQL's advanced data generation functions. For example, the following script was utilized to rapidly generate the customer base:

```
INSERT INTO customers (full_name, email, phone, address)
SELECT
    'Customer ' || i,
    'customer' || i || '@example.com',
    '555-' || LPAD((i % 10000)::text, 4, '0'),
    'Address ' || i
FROM generate_series(1, 1000) AS i;
```

(Similar generation scripts were executed for the remaining 5 entities to reach a total of 50,000 transactional records).

Implementation Proof: The following query validates the successful insertion of the high-volume dataset across all 6 entities:

```
SELECT 'customers' AS table_name, COUNT(*) FROM customers
UNION ALL
SELECT 'suppliers', COUNT(*) FROM suppliers
UNION ALL
SELECT 'items', COUNT(*) FROM items
```

UNION ALL

SELECT 'invoices', COUNT(*) FROM invoices

UNION ALL

SELECT 'invoice_items', COUNT(*) FROM invoice_items

UNION ALL

SELECT 'payments', COUNT(*) FROM payments;

